

# Seven design questions, answered against the code

2026-09-10 · answered from SIH26055 as it stands at commit 46d8449.

Every claim below is either a line of code in this repository, a number measured by a command run while writing this, or an artefact on disk that is named. Where a question assumes a component this system does not have, that is said plainly rather than answered around — three of the seven do, and the honest answer is more useful than a constructed one.

**Summary of what the seven questions turn out to be asking about.** Four of them (belief state, RL algorithm, reward, simulation numbers) are about the **learning lane** — rungs 7/8/9. Two (periodicity test, reversion trigger) are about the **published-baseline lane** — rung 6, our adaptation of Apfeld, Charlish and Koch. Those are different subsystems with different architectures, and the questions read as though they describe one system. They do not.

## 1. Belief-state representation

**There is no belief state in the probabilistic sense. No posterior, no Bayesian filter, no distribution over emitter frequency or mode.**

What the agent receives each step is a flat vector of **146 floats**, built only from its own scan history. Verified live:

```
observation_space : Box(146, ), float32
action_space      : Discrete(36)
```

Its layout (rfenv/env.py: :ScanEnv.\_observation, slice constants in rfenv/baselines/guard.py):

| block         | width | quantity                                                             | upper bound |
|---------------|-------|----------------------------------------------------------------------|-------------|
| hit_rate      | 36    | declared hits ÷ slots looked, per band                               | 1.0         |
| visit_density | 36    | airtime share ÷ <i>fair</i> share                                    | 36.0        |
| staleness     | 36    | slots since last look ÷ 43, i.e. in reference sweeps                 | 13.95       |
| current_band  | 36    | one-hot of the band just dwelt on                                    | 1.0         |
| clock         | 1     | slots elapsed ÷ 600                                                  | 1.0         |
| measured_dbm  | 1     | last dwell's mean S + noise, clamped to [−120, −20] dBm and rescaled | 1.0         |

These are **frequentist sufficient statistics of the agent's own observations**, not beliefs about hidden state. `hit_rate` is a running empirical rate; `staleness` is a deterministic counter; `visit_density` is a bookkeeping ratio. Nothing is updated by a measurement model, and there is no prior. The agent has no representation of "this band probably contains an emitter with period T."

Three properties are deliberate and are worth stating because they constrain what a belief-state approach would have to respect if one were built:

- **The observation ships; the reward does not.** Training is offline and the policy is frozen before deployment, so the reward may read ground truth (Z, per-emitter levels, first-intercept times) and the observation may not — that is all a fielded receiver has (D19, D20, D34). Enforced mechanically: `baselines/guard.py::restrict` hands a scheduler only `{slot, time_s, band, dwell_slots, Y}` and a policy that reaches for a truth key raises `KeyError` in a test rather than quietly scoring well.
- **Every episode starts cold.** No threat library, no carry-over between scenarios, no prior on who transmits where (D20) — the problem statement's "*absence of prior reliable intelligence*" made literal. A Bayesian prior over emitter placement would need D20 reopened.
- **The box is deliberately not the unit interval** (D55). `visit_density` and `staleness` were originally normalised by the episode, which pinned their means at 0.0278 and 0.070 — the bottom tenth of their declared range. Rung 5 had to divide `staleness` back out by hand to work at all. That correction now lives in the environment, so 1.0 means "an equal cut of airtime" and "one full pass overdue" respectively.

**The observation is not frozen.** D42's freeze covers band geometry, the slot clock,  $\gamma$ ,  $\sigma$  and the dwell schedule; reward candidates and observation extensions are explicitly *outside* it. A belief-state block could be added — it would be an observation extension in the sense D34 leaves open. It would also invalidate every existing checkpoint, which has already happened four times (109 → 145 → 146 → 147 → 146).

---

## 2. RL algorithm

---

Three, all standard deep RL from Stable-Baselines3. No Whittle index, no tabular method, no Q-learning in the tabular sense.

| rung | algorithm     | policy        | source                                                                            |
|------|---------------|---------------|-----------------------------------------------------------------------------------|
| 7    | DQN           | MlpPolicy     | <code>stable_baselines3.DQN</code> ,<br><code>rfenv/rl/dqn.py</code>              |
| 8    | PPO           | MlpPolicy     | <code>stable_baselines3.PPO</code> ,<br><code>rfenv/rl/ppo.py</code>              |
| 9    | Recurrent PPO | MlpLstmPolicy | <code>sb3_contrib.RecurrentPPO</code> ,<br><code>rfenv/rl/recurrent_ppo.py</code> |

Rung 9 is the only one carrying state across the episode: an LSTM hidden state inside the policy, so the action can in principle depend on the whole scan history rather than the latest look alone. That hidden state is the closest thing in this system to a "belief" — but it is a learned recurrent activation with no probabilistic semantics, not a posterior.

All three are deliberately untuned first passes: library-default hyperparameters except where a run's manifest records otherwise.

Two implementation details that materially affect results and are easy to get wrong:

- **Inference samples the policy; it does not take the argmax** (D54). Every RL result in this repository before 2026-09-09 was produced by taking the mode, and that was wrong for an on-policy algorithm: measured on one checkpoint, the action distribution had mean entropy 2.369 against  $\ln 36 = 3.584$  — broad, not collapsed — while its argmax sat on one band for **580 of 586 steps**. Sampling the same weights visits 31 of 36 bands and triples coverage. Rung 7 is the deliberate exception, since a DQN's greedy action *is* its policy.

- **torch's global generator is seeded per rung**, because SB3 samples from it and accepts no generator argument. Without that, a sampled rung would give different actions on re-runs at the same seed and the evaluation protocol's reproducibility guarantee would silently stop holding.

**Currently loadable: none of them.** Every DQN and PPO checkpoint predates an observation-width change and cannot execute; the rung-9 family is the only one with live checkpoints.

### 3. Reward function

Four registered candidates. Verified live:

```
REWARDS : ['explore', 'greedy', 'reward_balance', 'weighted']
default : reward_balance
```

None of them is detection probability or information gain. P\_d and P\_fa are frozen receiver properties and **no reward can move them** (D15, D21) — a false-alarm penalty prices a wasted dwell, nothing more.

The default, `reward_balance`, is a weighted combination of four terms, priced against the agent's own observation rather than against truth:

```
+0.5 · hit_rate[a]                exploit what has paid out
+1.0 · (1.5 - visit_density[a]) · stale  favour bands under-visited AND overdue
+0.5 · Z.sum()                    real occupancy this dwell
-3.0 · visit_density[a] · n_slots      cost on airtime already concentrated here
```

The other three are a deliberately constructed axis (D57): `greedy` is the pure-exploit corner (declarations plus remembered hit rate, no explore term, no camping cost), `explore` is the pure- explore corner (staleness minus airtime concentration plus new emitter-band discoveries), and `weighted` is a single knob  $\alpha$  between them, registered at  $\alpha = 0.3$ .

**What is being maximised is not obvious, and that is the central open problem.** The two headline metrics sit on opposite sides of the detection threshold — censored intercept time requires a declaration ( $Y = 1$ ), interception ratio is threshold-free — so **no single reward is aligned with both** (D28). That is why there is a candidate set and a selection rule rather than one reward.

#### The screen every candidate must now pass (D62)

Score four fixed ladder policies under a candidate — rung 2 (round-robin), rung 4 (camper), rung 5 (recency), rung 6a (Apfeld Active RFs) — over 8 seeds, paired per seed. A candidate passes only if it ranks rung 5 clearly above rung 2 relative to seed noise, and rung 4 clearly below both. No agent needed; minutes on a CPU. Measured:

| candidate                   | rung 2 | rung 5 | rung 4<br>(camper) | separation                     | camper<br>margin | verdict     |
|-----------------------------|--------|--------|--------------------|--------------------------------|------------------|-------------|
| <code>reward_balance</code> | 217.9  | 276.9  | <b>-414.1</b>      | 3.0 $\sigma$                   | 7.3 $\sigma$     | <b>PASS</b> |
| <code>weighted</code>       | 1455.7 | 1817.2 | 1323.0             | 2.6 $\sigma$                   | 0.4 $\sigma$     | FAIL        |
| <code>greedy</code>         | 459.0  | 776.6  | <b>+1198.3</b>     | 2.7 $\sigma$                   | -2.7 $\sigma$    | FAIL        |
| <code>explore</code>        | 1277.0 | 1238.1 | -205.2             | <b>-2.5<math>\sigma</math></b> | 31.5 $\sigma$    | FAIL        |

**reward\_balance is the only survivor.** The reasoning behind the screen: if a reward cannot rank two fixed, known policies in the order the evaluation metrics rank them, no agent trained on it can be expected to discover the difference either. Gradient descent optimises the reward it is given; it cannot recover an ordering the reward does not encode.

**This screen retired two candidates that had been live for a week.** `hit_z` (+1 per true occupied cell) and `hit_y` (+1 per declared hit) — the project's two original candidates — both rank the camper *above* every sweeping policy, at  $-2.3\sigma$  and  $-1.9\sigma$ . That is the explore/exploit tension stated in reward form: a reward that pays only for hits pays most for parking on the densest band. Every DQN and PPO run in this repository trained on one of those two. Both were removed from the registry entirely (D63).

The formal selection rule between surviving candidates (D47: the candidate beating round-robin on both headline metrics on the most paired episodes) **has still never been run.**

---

## 4. Periodicity detection test

---

**This exists — in rung 6, the published-baseline lane, not in the RL agent.** The RL rungs have no periodicity machinery of any kind.

Rung 6 is our adaptation of Apfeld, Charlish and Koch's adaptive search strategy. Declaring a band periodic is a **two-stage** procedure (`rfenv/baselines/apfeld.py`).

**Stage 1 — the point estimate.** For a band's binary declaration series `f` (zero wherever the receiver was tuned elsewhere), take the autocorrelation and return the lag of its largest non-zero-lag peak:

```
full      = np.correlate(f, f, mode="full")
positive = full[len(f):][:max_lag]      # lags 1 .. max_lag
return float(np.argmax(positive) + 1)
```

with `max_lag = 0.5 × len(history)` — a period estimate needs two periods of evidence to be an estimate rather than an artefact of the series' length. It returns `None` (no estimate) when the series holds fewer than two declarations, or is too short for a lag to mean anything.

**Be precise about what this is: there is no statistical test here.** No null hypothesis, no p-value, no peak-prominence or peak-to-sidelobe criterion, no confidence measure. It is an `argmax` over positive lags, guarded by two structural conditions. A series with no periodic structure still returns *some* lag — whichever bin happens to be largest.

**Stage 2 — the stability threshold, which is what actually declares a band periodic.** Keep the last `j` estimates for the band and declare the period stable when they agree:

```
hist.append(p_hat)
del hist[:-self.p.j]
if len(hist) < self.p.j or float(np.std(hist)) >= self.p.t_std_slots:
    return # not yet stable
self.stable[band] = float(np.mean(hist)) # declared periodic
```

**The threshold is `std(last 3 estimates) < 1.0 slot`, i.e. `j = 3`, `T_std = 1.0`.**

Both constants are ours, not the paper's — Apfeld names every parameter and fixes none. Their justifications are deliberately *not* the score they produce: `j = 3` is the smallest count for which a standard deviation means anything, and `T_std = 1 slot` is the clock's own resolution, so "stable" means "agreeing to the resolution of the instrument" rather than to a tuned tolerance.

So the honest characterisation: **agreement-based, not significance-based.** Three consecutive autocorrelation peaks landing within one slot of each other is treated as evidence of periodicity. That is a reasonable engineering heuristic and it is *not* a

hypothesis test, and nothing in this repository currently measures its false-positive rate.

## 5. Reversion trigger

**Also exists, also in rung 6 only.** The signal is **missed detections on scheduled visits**, and the threshold is **2 consecutive**.

Once a band's period is declared stable it leaves the exploration phase and is scheduled explicitly — dwells anchored at the last detection and repeating at multiples of the estimated period, with a  $\pm 1$ -slot window. That scheduled-revisit mode is the closest thing in this system to "phase-sync": the receiver is no longer searching that band, it is keeping an appointment.

Reversion out of it:

```
elif self._from_schedule:
    self.misses[band] = self.misses.get(band, 0) + 1
    if self.misses[band] >= self.p.s:          # s = 2
        self._forget(band)
```

Two details matter and are easy to misread:

- **Only a *scheduled visit* that finds nothing counts as a miss** (`self._from_schedule`). A dwell that lands on the band for any other reason and declares nothing does not increment the counter. The trigger is specifically "the appointment was kept and the emitter was not there."
- **`_forget` is a full reset, not a decay.** It drops the stable period, clears the visit schedule, discards the estimate history entirely, zeroes the miss counter, and returns the band to the tentative list — back to exploration, with no memory of the period it had inferred.

**There is no confidence score anywhere in this.** No posterior, no decaying trust, no soft handover. It is a two-strike counter on a boolean condition. `s = 2` is ours: the paper works on 5-minute episodes and ours are 30 seconds, so a long patience would spend the whole episode wrong.

### The measured verdict on this whole mechanism

**Rung 6's period-estimation half measurably hurts** on our episodes (D45). Apfeld's own ablation — the same strategy with period estimation disabled, "Active RFs", registered as rung 6a — is strictly better on this problem:

| rung                                   | interception ratio | censored intercept time | emitter coverage | beats round-robin on both |
|----------------------------------------|--------------------|-------------------------|------------------|---------------------------|
| 6a Active RFs (no period estimation)   | 0.1320             | 4.32 s                  | 0.860            | 53.2%                     |
| 6 full Apfeld (with period estimation) | 0.2455             | 14.86 s                 | 0.367            | 4.1%                      |

1.9× the interception ratio for 3.4× the intercept time and less than half the coverage. The phase-locked schedule concentrates airtime on bands it believes it understands, and on a 30-second binary-detection episode that costs more than it buys. This is our adaptation on our problem, not the paper's result on the paper's problem — the paper's episodes are ten times longer and its receiver reports SNR rather than a binary declaration.

## 6. Simulation numbers

**Yes — extensive, and all reproducible from the repository.** This is the best-evidenced of the seven questions. Three separate bodies of measurement exist.

### 6.1 Environment validation gates

Four gates, criteria written into `rfenv/validate.py`: **GATES before** the first run and asserted against the decision record by a test, because a threshold chosen once the measurement is visible is not a gate. Run 2026-09-04 over 47 configs, seed 0.

| gate | what it checks                                                                                                                                                                 | result                                                                                                                                                                                                          |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Out-of-sample prediction: build truth from <i>stare</i> only, replay Turing's scan schedule, score per dwell against the raw scan recordings — data never used in construction | <b>MEASURED</b> (deliberately not gated): accuracy <b>0.8585</b> , precision <b>0.8819</b> , recall <b>0.6932</b> , MCC <b>0.6854</b> , per-band r <b>0.935</b> , over 23,594 dwells against a 0.3540 base rate |
| 2    | Per-band occupancy reconstruction                                                                                                                                              | <b>PASS</b>                                                                                                                                                                                                     |
| 3    | Theory: controlled periodic case against Köksal's closed forms                                                                                                                 | <b>PASS</b> — <b>P██(T█)</b> <b>0.18333</b> against reference <b>0.18333</b> ( $ \Delta  0.0000$ ); max first intercept 12.90 s $\geq$ 10.75 s                                                                  |
| 4    | Extremes: 12 structural assertions on a 2-emitter and a 99-emitter scenario                                                                                                    | <b>PASS</b> , 12/12                                                                                                                                                                                             |

Gate 1 is the only one whose two sides use different data, and it is the one that matters most: it is a genuine out-of-sample prediction rather than a fit. Its threshold is deliberately undecided — inventing one after seeing 0.8585 would be exactly the failure the gate machinery exists to prevent.

A known limitation, recorded rather than hidden: band 0 (250 MHz) is 58.97% occupied in the recordings and 0.00% predicted, because the stare recordings cannot see below 500 MHz.

### 6.2 The scheduler ladder — measured, 2,223 episodes

`python -m rfenv.compare --seeds 3 --sampled 10`, 57 scenarios  $\times$  3 seeds, artefacts in `runs/acceptance_2026-09-10/`:

| rung | scheduler                   | interception ratio | censored intercept time (s) | emitter coverage |
|------|-----------------------------|--------------------|-----------------------------|------------------|
| 1    | random                      | 0.0669             | 4.16                        | 0.858            |
| 2    | round-robin (equal airtime) | 0.0605             | 4.18                        | 0.865            |
| 3    | Turing reference sweep      | 0.0805             | 3.74                        | 0.864            |
| 4    | greedy camper               | 0.2088             | 9.67                        | 0.497            |

| rung | scheduler                                | interception ratio | censored intercept time (s) | emitter coverage |
|------|------------------------------------------|--------------------|-----------------------------|------------------|
| 5    | <b>recency / activity</b>                | <b>0.1105</b>      | <b>3.34</b>                 | <b>0.887</b>     |
| 6a   | Apfeld Active RFs                        | 0.1320             | 4.32                        | 0.860            |
| 6    | Apfeld adaptive                          | 0.2455             | 14.86                       | 0.367            |
| —    | camper, truth-fed<br>(reference line)    | 0.5680             | 15.71                       | 0.296            |
| —    | pulse-capture oracle<br>(reference line) | 0.6579             | 8.01                        | 0.691            |

**Scan efficiency against the baseline is measured paired, per episode**, not as a difference of means — scenario difficulty spans 2 to 99 emitters and a mean can clear a mean while losing most scenarios. Rung 5 Pareto-dominates round-robin on **67.3%** of episodes. That, not round-robin, is the bar the learning lane has to clear.

### 6.3 The learning lane — and the honest status of its numbers

Latest run, `runs/smoke_final_all_scenarios/`, 47 scenarios  $\times$  1 seed, one training run snapshotted every 100k steps, **paired against rung 5**:

| checkpoint  | ratio  | cTTI (s)    | coverage | Pareto-dominates rung 5 |
|-------------|--------|-------------|----------|-------------------------|
| 100k        | 0.1247 | 3.04        | 0.911    | 27.7%                   |
| 200k        | 0.1259 | 2.52        | 0.933    | 36.2%                   |
| <b>300k</b> | 0.1172 | <b>2.43</b> | 0.917    | <b>42.6%</b>            |
| 400k        | 0.0960 | 2.75        | 0.905    | 23.4%                   |
| 500k        | 0.0824 | 2.70        | 0.904    | 12.8%                   |
| 600k        | 0.0960 | 3.12        | 0.915    | 29.8%                   |
| 700k        | 0.0850 | 3.03        | 0.899    | 6.4%                    |
| 800k        | 0.0985 | 3.20        | 0.896    | 17.0%                   |
| 900k        | 0.0983 | 2.83        | 0.895    | 25.5%                   |
| 1M          | 0.0922 | 3.41        | 0.889    | 8.5%                    |

**Convergence: performance peaks around 200–300k steps and degrades thereafter.** The 1M-step model is roughly a fifth as good against the bar as the 300k one. Anyone reporting "the 1M model" as the result would be reporting the worst end of a curve that peaked early.

## Two caveats that decide how these numbers may be used:

1. **No RL result in this repository is currently clean.** Until 2026-09-10 training sampled emitters from all 47 development configs while evaluation ran those same 47 — and the baselines do not train, so the asymmetry ran one way, ours. The split is now closed: 35 training / 12 validation configs by a rule written before anyone looked at which landed where, with the pool rebuilt from the training half — **2,600 contributions / 1,431 emitters against 843 / 482, zero shared.** Every checkpoint above predates that split. The numbers are arithmetically sound and the comparison is contaminated; they are **not** written into the evaluation record as a result.
2. **Checkpoint selection is pre-registered** (D61) precisely because of the curve above: highest  $P(\text{dominates rung 5}) - P(\text{dominated by rung 5})$  on the validation half, ties toward fewer steps, fixed in code before the run that uses it. Picking the best of ten checkpoints after seeing evaluation numbers fits the evaluation set through the choice, and the bias grows every iteration.

**The 45 held-out test pairs remain untouched.** They are mechanically refused without an explicit flag, and they are spent once, on a result we are confident in.

---

## 7. Phase-lock ceiling

**There is no phase-lock efficiency-ceiling claim in this repository — neither derived nor estimated.** No such quantity is computed, asserted or recorded anywhere. If it exists, it is in a document that is not in this repo, and nothing here should be cited as supporting it.

What does exist, and what the question may be reaching for, are **three different things that all behave like ceilings**, with sharply different epistemic status. The distinction matters:

**Derived, and genuinely closed-form.** Köksal's interception probability  $P_{\text{K}}(T)$  for a periodic emitter against a periodic receiver — gate 3 above. Our environment reproduces the discrete reference to  $|\Delta| = 0.0000$ . But note the standing caveat: the extension  $P_{\text{K}}(T) = 1 - [1 - P_{\text{K}}(T)]^{T/T}$  **is reported and never gated**, because it assumes successive receiver periods are independent, which is false for a deterministic periodic pair. Gating on it would fail a correct environment. It is also explicitly an environment check and never a scheduler claim — Köksal assumes prior emitter knowledge, which this system's cold-start rule refuses.

**Measured, not derived.** The two reference lines: `oracle_pulse` (0.6579 interception ratio) and `camper_oracle` (0.5680). Both read the ground-truth grid and are not schedulers. Neither is a bound in any mathematical sense — they are what specific truth-reading strategies happened to score. And a reference line is not a competitor: `oracle_pulse` is a ceiling for interception ratio **only**, and measured, it *loses* censored intercept time to plain round-robin on 80.7% of episodes.

**Measured, and specifically shown not to be reachable.** `camper_oracle` scores 0.568 because it knows where the pulses are; the observation-fed camper — the same strategy denied truth — gets 0.209. Illumination density is truth-side, and binary declarations are a poor proxy for it. The gap between those two numbers is the price of the deployability constraint, and it is measured rather than assumed.

**So if a "phase-lock ceiling" is to be claimed, none of the above supplies it.** The nearest honest statement this repository supports is the empirical one in §5: phase-locked scheduling (rung 6) is *worse* than the same strategy without it on 30-second binary-detection episodes, by a factor of 3.4 on intercept time. That is a measured result on our adaptation, not a ceiling, and it points the opposite way from an efficiency argument for phase-sync.

---

## What this system does not have

Stated once, together, because three of the seven questions assume them:

- **No Bayesian belief state, no filter, no posterior over emitter parameters.** The observation is frequentist sufficient statistics of the agent's own looks.
- **No Whittle index, no restless-bandit index policy.** The problem is genuinely a restless bandit and an index policy is a reasonable thing to want; nothing of the sort is implemented.
- **No mode-switching architecture in the learning lane.** No phase-sync mode, no belief-state mode, nothing to revert between. Rung 6 has a two-state exploration/scheduled structure, and it is a hand-written baseline from a paper, not the agent.
- **No phase-lock ceiling claim, derived or estimated.**

None of these is ruled out by the frozen core — the freeze covers band geometry, the slot clock and the receiver constants, and explicitly leaves reward candidates and observation extensions open. Any of them could be built. None has been, and this document should not be read as evidence that any of them has.

---

## Reproducing every number above

```
python -m rfenv.validate # the four gates (§6.1)
python -m rfenv.compare --seeds 3 --sampled 10 --figures # the ladder (§6.2, §6.3)
python -m rfenv.reward_gate # the reward screen (§3)
python -m rfenv.split # the train/validation split (§6.3)
python -m rfenv.selection runs/checkpoints/<run>_*.zip # pre-registered selection (§6.3)
```

Source of record for the reasoning behind each decision cited by number: `docs/project/DECISIONS.md`. Metrics and protocol: `docs/project/EVALUATION.md`.